

CSCI 5380

Network Virtualization and Orchestration

Lab 5

P4 - Programming Protocol-Independent Packet Processors

University of Colorado Boulder
Department of Computer Science
Network Engineering

Professor Levi Perigo, Ph.D.

Summary

P4 is a programming language designed to allow programming of packet forwarding planes. In contrast to a general-purpose language such as C or Python, P4 is a domain-specific language with several constructs optimized around network data forwarding. P4 is an open-source, permissively licensed language and is maintained by a non-profit organization called the P4 Language Consortium. The purpose of this lab is to get an introductory understanding of P4 programs in a virtual environment by implementing traditional and custom network functions. The objectives of this lab are to be used as guidelines, and additional exploration by the student is strongly encouraged.

Objective 1: Setting up environment

1. Download the P4 VM from <https://github.com/jafingerhut/p4-guide/blob/master/bin/README-install-troubleshooting.md> and follow the installation instructions.
2. Clone the following repository inside the P4 VM: <https://github.com/p4lang/tutorials>
3. Reinitialize the MAC address of the NAT adapter.
4. Login for the VM is p4:p4.

Objective 2: P4 programming

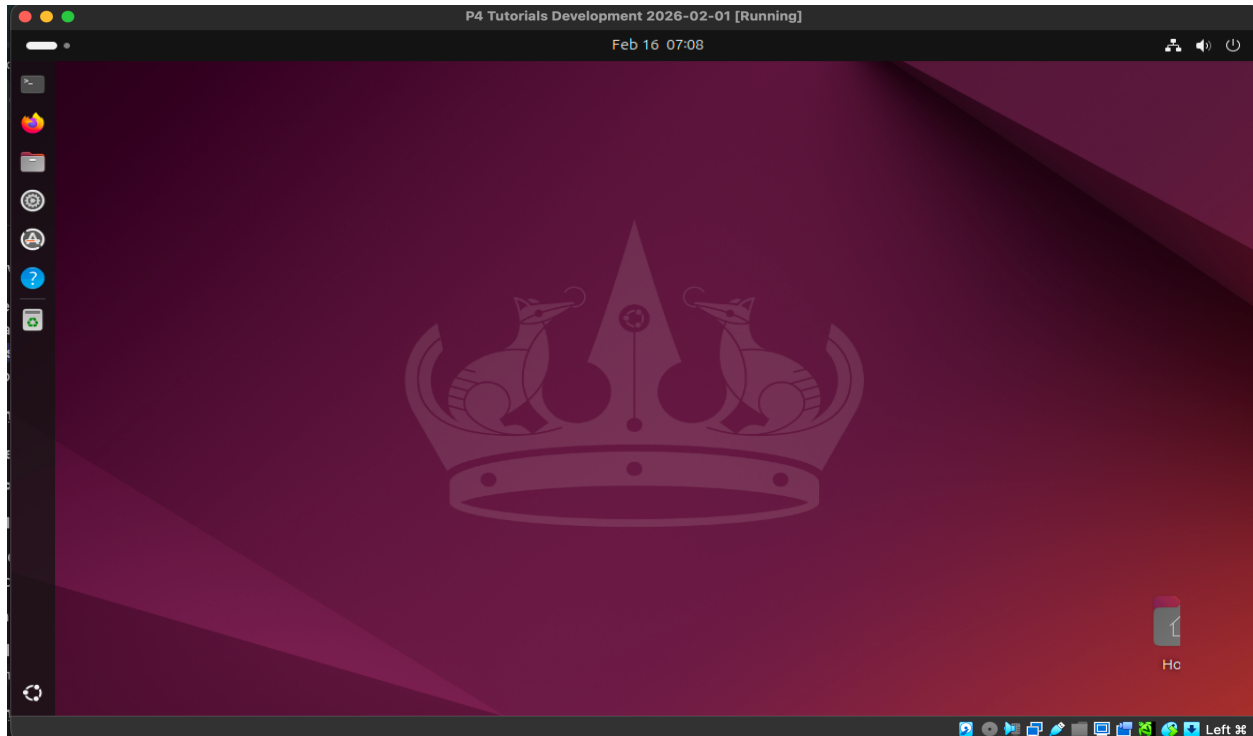
1. Navigate to the directory tutorials/exercises.
2. There are 8 objectives to complete present in below directories –
 - i. basic
 - ii. basic_tunnel
 - iii. calc
 - iv. ecn
 - v. load_balance
 - vi. mri
 - vii. p4runtime
 - viii. source_routing
3. For each objective, first read the README.md file present in each directory to find out what needs to be done. Use those instructions to complete the blanks in the .p4 file in each directory.
4. The steps to execute and test the code are also mentioned in the README.md file.
5. There is a reference solution in each of the 'solution' sub-directory. Feel free to compare your implementation to the reference.

Deliverable

- Write the steps to complete each objective. Also, write what is the difference of completing each objective using P4 instead of using traditional or OpenFlow networking concepts.
- Create something new (your choice) in addition to the base objectives
 - Submit a simple explanation of what you created

Objective 1 – Setting up environment.

Downloaded the Ova file from the link in Obj1 and then performed the File > Import > .ova file and I was able to spin up a VM for P4-



Navigate to Tutorials Folder –

```
[p4@p4dev:~/tutorials]
[07:13:17] $ ls -al
total 8208
 4 drwxr-xr-x  9 p4 p4    4096 Feb  1 15:12 ./
 4 drwxr-x--- 15 p4 p4    4096 Feb 16 07:06 ../
 4 -rw-r--r--  1 p4 p4   2505 Feb  1 15:12 CONTRIBUTING.md
 4 drwxr-xr-x  2 p4 p4    4096 Feb  1 15:12 doc/
 4 drwxr-xr-x 15 p4 p4    4096 Feb  1 15:12 exercises/
 4 drwxr-xr-x  8 p4 p4    4096 Feb  1 15:12 .git/
 4 -rw-r--r--  1 p4 p4     321 Feb  1 15:12 .gitignore
12 -rw-r--r--  1 p4 p4   10317 Feb  1 15:12 LICENSE
36 -rw-r--r--  1 p4 p4   34052 Feb  1 15:12 p4-cheat-sheet.pdf
8104 -rw-r--r--  1 p4 p4 8298297 Feb  1 15:12 P4_tutorial.pdf
12 -rw-r--r--  1 p4 p4   11899 Feb  1 15:12 README.md
 4 drwxr-xr-x  6 p4 p4    4096 Feb  1 15:12 utils/
 4 drwxr-xr-x  2 p4 p4    4096 Feb  1 15:12 vm/
 4 drwxr-xr-x  3 p4 p4    4096 Feb  1 15:12 vm-ubuntu-20.04/
 4 drwxr-xr-x  3 p4 p4    4096 Feb  1 15:12 vm-ubuntu-24.04/

[p4@p4dev:~/tutorials]
[07:13:18] $
```

Objective 2 – P4 Programming

Exercise directory –

```
[p4@p4dev:~/tutorials/exercises]
[07:16:48] $ ls -al
total 60
4 drwxr-xr-x 15 p4 p4 4096 Feb  1 15:12 ./
4 drwxr-xr-x  9 p4 p4 4096 Feb  1 15:12 ../
4 drwxr-xr-x  5 p4 p4 4096 Feb  1 15:12 basic/
4 drwxr-xr-x  3 p4 p4 4096 Feb  1 15:12 basic_tunnel/
4 drwxr-xr-x  3 p4 p4 4096 Feb  1 15:12 calc/
4 drwxr-xr-x  3 p4 p4 4096 Feb  1 15:12 ecn/
4 drwxr-xr-x  4 p4 p4 4096 Feb  1 15:12 firewall/
4 drwxr-xr-x  3 p4 p4 4096 Feb  1 15:12 flowcache/
4 drwxr-xr-x  4 p4 p4 4096 Feb  1 15:12 link_monitor/
4 drwxr-xr-x  3 p4 p4 4096 Feb  1 15:12 load_balance/
4 drwxr-xr-x  3 p4 p4 4096 Feb  1 15:12 mri/
4 drwxr-xr-x  4 p4 p4 4096 Feb  1 15:12 multicast/
4 drwxr-xr-x  3 p4 p4 4096 Feb  1 15:12 p4runtime/
4 drwxr-xr-x  3 p4 p4 4096 Feb  1 15:12 qos/
4 drwxr-xr-x  3 p4 p4 4096 Feb  1 15:12 source_routing/
```

1. Basic –

Once I read the README.MD file - I ran the make run from CLI which deployed the topology-

```
s1 -> gRPC port: 50051
s2 -> gRPC port: 50052
s3 -> gRPC port: 50053
s4 -> gRPC port: 50054
*****
h1
default interface: eth0 10.0.1.1          08:00:00:00:01:11
*****
*****
h2
default interface: eth0 10.0.2.2          08:00:00:00:02:22
*****
*****
h3
default interface: eth0 10.0.3.3          08:00:00:00:03:33
*****
*****
h4
default interface: eth0 10.0.4.4          08:00:00:00:04:44
*****
Starting mininet CLI
```

Based on code review of the basic.p4 – I could there were some of the TODO marked in code which needed to be completed to allow for this to work – I added the parsing logic by referring to comments in the code as follows -

```

parser MyParser(packet_in packet,
                out headers hdr,
                inout metadata meta,
                inout standard_metadata_t standard_metadata) {

    state start {
        /* TODO: add parser logic
        * Suggested outline:
        * 1) Extract Ethernet: packet.extract(hdr.ethernet);
        * 2) If hdr.ethernet.etherType == TYPE_IPV4 -> parse IPv4
        * 3) Otherwise -> transition accept
        */

        packet.extract(hdr.ethernet);
        transition select(hdr.ethernet.etherType) {
            TYPE_IPV4: parse_ipv4;
            default: accept;
        }
    }

    state parse_ipv4 {
        packet.extract(hdr.ipv4);
        transition accept;
    }
}

```

Similarly, for ipv4 forward action – I added the lines mentioned in the comment -

```

action ipv4_forward(macAddr_t dstAddr, egressSpec_t port) {
    /*
    Action function for forwarding IPv4 packets.

    TODO: Implement the forwarding steps, for example:
    - standard_metadata.egress_spec = port;
    - hdr.ethernet.dstAddr = dstAddr;
    - (optionally) set hdr.ethernet.srcAddr to the switch MAC for 'port'
    - adjust IPv4 TTL and checksums as needed
    */
    // Set output port
    standard_metadata.egress_spec = port;

    // Rewrite MAC addresses
    hdr.ethernet.dstAddr = dstAddr;
    // (optional) keep srcAddr unchanged for simplicity

    // Decrement TTL
    hdr.ipv4.ttl = hdr.ipv4.ttl - 1;
}

```

Validation check of the packet is valid ipv4 before applying ipv4_lpm function -

```

apply {
    /* TODO: fix ingress control logic
    * - Good practice: apply ipv4_lpm only when the IPv4 header is valid, e.g.:
    *   if (hdr.ipv4.isValid()) { ipv4_lpm.apply(); }
    * This skeleton currently applies unconditionally for the exercise.
    */

    if (hdr.ipv4.isValid()) {
        ipv4_lpm.apply();
    }
}

control MyDeparser(packet_out packet, in headers hdr) {
    apply {
        /*
        Typical implementation (left as a TODO for learners):
        packet.emit(hdr.ethernet);
        packet.emit(hdr.ipv4); // per P4_16 spec, emit appends a header
                                // only if it is valid; no 'if' needed.
        */
        packet.emit(hdr.ethernet);
        packet.emit(hdr.ipv4);
    }
}

```

Once all the TODO sections were complete – I performed the make run and performed the pingall actions –

```
s1 -> gRPC port: 50051
s2 -> gRPC port: 50052
s3 -> gRPC port: 50053
s4 -> gRPC port: 50054
*****
h1
default interface: eth0 10.0.1.1      08:00:00:00:01:11
*****
*****
h2
default interface: eth0 10.0.2.2      08:00:00:00:02:22
*****
*****
h3
default interface: eth0 10.0.3.3      08:00:00:00:03:33
*****
*****
h4
default interface: eth0 10.0.4.4      08:00:00:00:04:44
*****
Starting mininet CLI
```

```
mininet> pingall
*** Ping: testing ping reachability
h1 -> h2 h3 h4
h2 -> h1 h3 h4
h3 -> h1 h2 h4
h4 -> h1 h2 h3
*** Results: 0% dropped (12/12 received)
mininet> █
```


2. Basic Tunnel–

Updated the Parser to allow for packet parsing based on ipv4 packets as well as tunnel type packets –

```
/****** P A R S E R *****/
*****/

// TODO: Update the parser to parse the myTunnel header as well
parser MyParser(packet_in packet,
                out headers hdr,
                inout metadata meta,
                inout standard_metadata_t standard_metadata) {

    state start {
        transition parse_ethernet;
    }

    state parse_ethernet {
        packet.extract(hdr.ethernet);
        transition select(hdr.ethernet.etherType) {
            TYPE_MYTUNNEL : parse_mytunnel;
            TYPE_IPV4      : parse_ipv4;
            default        : accept;
        }
    }

    state parse_mytunnel {
        packet.extract(hdr.myTunnel);
        transition select(hdr.myTunnel.proto_id) {
            TYPE_IPV4 : parse_ipv4;
            default   : accept;
        }
    }

    state parse_ipv4 {
        packet.extract(hdr.ipv4);
        transition accept;
    }
}
```

Added todo for tunnel forwarding logic based on the destination id and control flow based on if tunnel is valid -

```
// TODO: declare a new action: myTunnel_forward(egressSpec_t port)
action myTunnel_forward(egressSpec_t port) {
    standard_metadata.egress_spec = port;
}

// TODO: declare a new table: myTunnel_exact
// TODO: also remember to add table entries!
table myTunnel_exact {
    key = {
        hdr.myTunnel.dst_id: exact;
    }
    actions = {
        myTunnel_forward;
        drop;
        NoAction;
    }
    size = 1024;
    default_action = drop();
}

apply {
    // TODO: Update control flow
    if (hdr.myTunnel.isValid()) {
        myTunnel_exact.apply();
    } else if (hdr.ipv4.isValid()) {
        ipv4_lpm.apply();
    }
}
```

Emmit the packet for ethernet and mytunnel along with ipv4 attributes -

```

/*****
***** D E P A R S E R *****/
*****/

control MyDeparser(packet_out packet, in headers hdr) {
    apply {
        packet.emit(hdr.ethernet);
        // TODO: emit myTunnel header as well
        packet.emit(hdr.myTunnel);
        packet.emit(hdr.ipv4);
    }
}
}
```

Performed Make Run -

```

*****
h1
default interface: eth0 10.0.1.1          08:00:00:00:01:11
*****
h2
default interface: eth0 10.0.2.2          08:00:00:00:02:22
*****
h3
default interface: eth0 10.0.3.3          08:00:00:00:03:33
*****
Starting mininet CLI
```

Without tunneling -

```

mininet> h1 python3 send.py 10.0.2.2 "p4 testing"
sending on interface eth0 to IP addr 10.0.2.2
#### Ethernet ####
dst      = ff:ff:ff:ff:ff:ff
src      = 08:00:00:00:01:11
type     = IPv4
#### IP ####
version  = 4
ihl      = 5
tos      = 0x0
len      = 50
id       = 1
flags    = 
frag     = 0
ttl      = 64
proto    = tcp
chksum   = 0x63c3
src      = 10.0.1.1
dst      = 10.0.2.2
\options \
#### TCP ####
sport    = 64206
dport    = 1234
seq      = 0
ack      = 0
dataofs  = 5
reserved = 0
flags    = S
window   = 8192
chksum   = 0xa048
urgptr   = 0
options  = []
#### Raw ####
load     = 'p4 testing'
```

With Tunneling(I could see the Tunnel header encapsulated before the IPV4 headers) -

```

mininet> h1 python3 send.py 10.0.2.2 "p4 testing" --dst_id 2
sending on interface eth0 to dst_id 2
#### Ethernet ####
dst      = ff:ff:ff:ff:ff:ff
src      = 08:00:00:00:01:11
type     = 0x1212
#### MyTunnel ####
pid      = 2048
dst_id   = 2
#### Raw ####
```

3. Calc –

Added the additional operation mentioned in the TODO comments –

```
header p4calc_t {  
    /* TODO  
    * fill p4calc_t header with P, four, ver, op, operand_a, operand_b, and res  
    entries based on above protocol header definition.  
    */  
    bit<8> p;  
    bit<8> four;  
    bit<8> ver;  
    bit<8> op;  
    bit<32> operand_a;  
    bit<32> operand_b;  
    bit<32> res;  
}
```

Performed the stat check and action block to convert mac address on match and send back to same port

```
state check_p4calc {  
    /* TODO: just uncomment the following parse block */  
    transition select(packet.lookahead<p4calc_t>().p,  
                      packet.lookahead<p4calc_t>().four,  
                      packet.lookahead<p4calc_t>().ver) {  
        (P4CALC_P, P4CALC_4, P4CALC_VER) : parse_p4calc;  
        default : accept;  
    }  
}
```

```
control MyIngress(inout headers hdr,  
                  inout metadata meta,  
                  inout standard_metadata_t standard_metadata) {  
    action send_back(bit<32> result) {  
        /* TODO  
        * - put the result back in hdr.p4calc.res  
        * - swap MAC addresses in hdr.ethernet.dstAddr and  
        *   hdr.ethernet.srcAddr using a temp variable  
        * - Send the packet back to the port it came from  
        *   by saving standard_metadata.ingress_port into  
        *   standard_metadata.egress_spec  
        */  
        hdr.p4calc.res = result;  
  
        bit<48> tmp;  
        tmp = hdr.ethernet.dstAddr;  
        hdr.ethernet.dstAddr = hdr.ethernet.srcAddr;  
        hdr.ethernet.srcAddr = tmp;  
  
        standard_metadata.egress_spec = standard_metadata.ingress_port;  
    }  
  
    action operation_add() {  
        /* TODO call send_back with operand_a + operand_b */  
        send_back(hdr.p4calc.operand_a + hdr.p4calc.operand_b);  
    }  
  
    action operation_sub() {  
        /* TODO call send_back with operand_a - operand_b */  
        send_back(hdr.p4calc.operand_a - hdr.p4calc.operand_b);  
    }  
  
    action operation_and() {  
        /* TODO call send_back with operand_a & operand_b */  
        send_back(hdr.p4calc.operand_a & hdr.p4calc.operand_b);  
    }  
}
```

Once completed, I performed make run –

```
*****
h1
default interface: eth0 10.0.1.1          08:00:00:00:01:01
*****
*****
h2
default interface: eth0 10.0.1.2          08:00:00:00:01:02
*****
Starting mininet CLI
```

Testing -

```
mininet> h1 python3 calc.py
/home/p4/tutorials/exercises/calc/calc.py:48: SyntaxWarning: invalid escape sequence '\s'
  pattern = "^s*([0-9]+)\s*"
/home/p4/tutorials/exercises/calc/calc.py:57: SyntaxWarning: invalid escape sequence '\s'
  pattern = "^s*([-\+&|^\s])\s*"
> 1+1
1+1
2
> 4+4
4+4
8
```

So this basically extracts the incoming packet data and perform this calculator function and returns it to same port where it came from to show the result.

4. ECN –

In this exercise, we will implement ecn based to indicate congestion which would allow us to ensure no packet drops, so we will detect the congestion based on queue depth and add 11 flag which is congestion detected –

```
/*
 * TODO: split tos to two fields 6 bit diffserv and 2 bit ecn
 */
header ipv4_t {
    bit<4>    version;
    bit<4>    ihl;

    bit<6>    diffserv;
    bit<2>    ecn;

    bit<16>   totalen;
    bit<16>   identification;
    bit<3>    flags;
    bit<13>   fragOffset;
    bit<8>    ttl;
    bit<8>    protocol;
    bit<16>   hdrChecksum;
    ip4Addr_t srcAddr;
    ip4Addr_t dstAddr;
}
```

we will compare the queue depth and based on the threshold we will set the ecn value -

```
control MyEgress(inout headers hdr,
                 inout metadata meta,
                 inout standard_metadata_t standard_metadata) {
    apply {
        /*
         * TODO:
         * - if ecn is 1 or 2
         * - compare standard_metadata.enq_qdepth with threshold
         * and set hdr.ipv4.ecn to 3 if larger
         */
        if (hdr.ipv4.isValid()) {
            if (hdr.ipv4.ecn == 1 || hdr.ipv4.ecn == 2) {
                if (standard_metadata.enq_qdepth > ECN_THRESHOLD) {
                    hdr.ipv4.ecn = 3;
                }
            }
        }
    }
}
```

```

control MyComputeChecksum(inout headers hdr, inout metadata meta) {
  apply {
    /* TODO: replace tos with diffserve and ecn */
    update_checksum(
      hdr.ipv4.isValid(),
      {
        hdr.ipv4.version,
        hdr.ipv4.ihl,
        hdr.ipv4.diffserv,
        hdr.ipv4.ecn,
        hdr.ipv4.totalLen,
        hdr.ipv4.identification,
        hdr.ipv4.flags,
        hdr.ipv4.fragOffset,
        hdr.ipv4.ttl,
        hdr.ipv4.protocol,
        hdr.ipv4.srcAddr,
        hdr.ipv4.dstAddr },
      hdr.ipv4.hdrChecksum,
      HashAlgorithm.csum16);
  }
}

```

Testing –

The image displays a collage of screenshots from a network testing environment. The top section shows a terminal window with instructions for running a switch CLI, including commands like `simple_switch_CLI --thrift-port <switch thrift port>` and `tail -f /home/p4/tutorials/exercises/ecn/logs/<switchname>.log`. Below this, a network monitor window shows a list of packets with their corresponding ECN values (0x1, 0x3). The bottom section shows a terminal window with the execution of a script to sniff on eth0, with output showing the sniffing process and the resulting log file.

We could see here the tos value is detected when the queue length becomes larger which means the ecn bit is detected.

5. Load balance –

In this we added 5-tuple based to generate the hash value which will be used to load balance the flows –

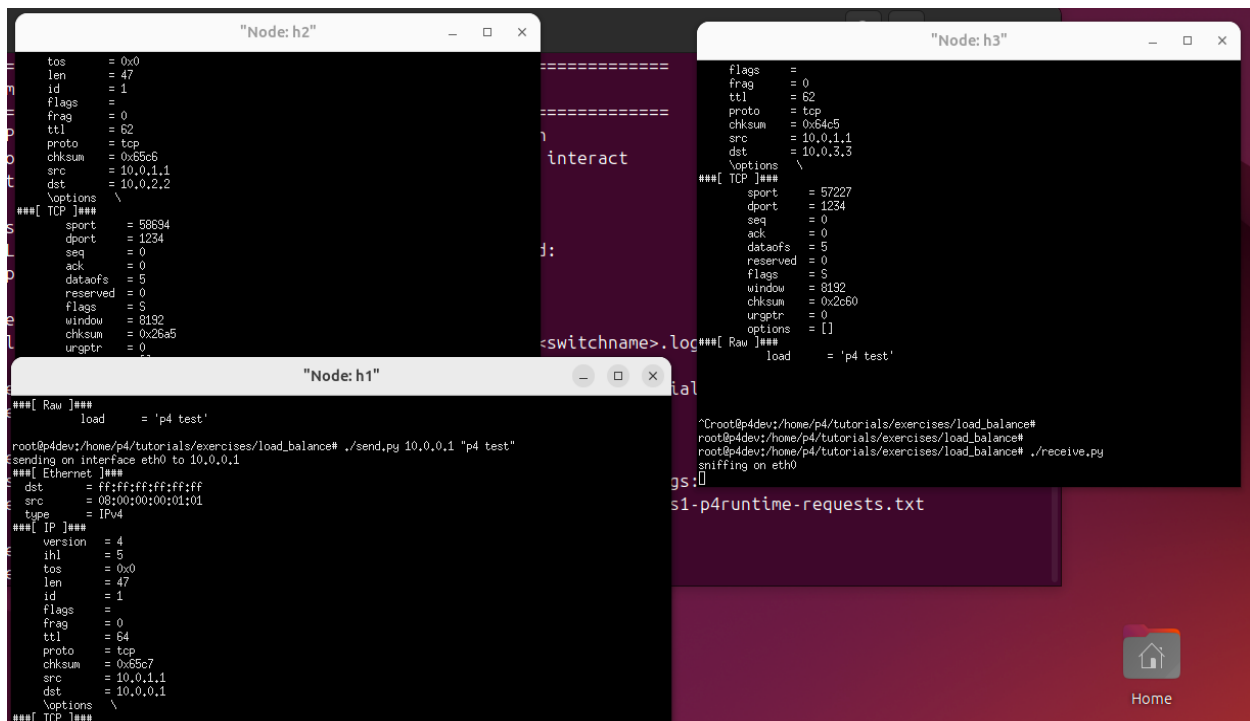
```
// --- ECMP hash computation ---
action set_ecmp_select(bit<16> ecmp_base, bit<32> ecmp_count) {
    // Compute a CRC16 over the 5-tuple and map to ECMP group
    hash(meta.ecmp_select, HashAlgorithm.crc16,
        { hdr.ipv4.srcAddr,
          hdr.ipv4.dstAddr,
          hdr.ipv4.protocol,
          hdr.tcp.srcPort,
          hdr.tcp.dstPort });

    // Map hash to [0, ecmp_count-1] and add base
    meta.ecmp_select = (bit<14>)((meta.ecmp_select % ecmp_count) + ecmp_base);
}
```

Checking for valid v4 packet type and ttl greater then 0 and applying the ecmp –

```
apply {
    /* TODO: apply ecmp_group table and ecmp_nhop table if IPv4 header is
     * valid and TTL hasn't reached zero
     */
    if (hdr.ipv4.isValid() && hdr.ipv4.ttl > 0) {
        ecmp_group.apply();
        ecmp_nhop.apply();
    }
}
```

Testing –We could see the packet is received on H2 or H3 which means load balancing works when I disable and enable the H3 to force the packet to go to H2.



6. MRI –

In This, we implement an MRI (Multi route inspection) enables to collect the metrics for the path through which the packet traverses -

```
state parse_ipv4_option {
    /*
    * TODO: Add logic to:
    * - Extract the ipv4_option header.
    * - If hdr.ipv4_option.option == IPV4_OPTION_MRI, transition to parse_mri.
    * - Otherwise, transition to accept.
    */
    transition accept;
}

state parse_mri {
    /*
    * TODO: Add logic to:
    * - Extract hdr.mri.
    * - Set meta.parser_metadata.remaining = hdr.mri.count
    * - If meta.parser_metadata.remaining == 0, transition to accept.
    * - Otherwise, transition to parse_swtrace.
    */
    transition accept;
}

state parse_swtrace {
    /*
    * TODO: Add logic to:
    * - Extract the next hdr.swtraces entry (hdr.swtraces[...]).
    * - Decrement meta.parser_metadata.remaining by 1.
    * - If meta.parser_metadata.remaining == 0, transition to accept.
    * - Otherwise, remain in parse_swtrace to extract next entry.
    */
    transition accept;
}
}
```

Adding the trace -

```
control MyEgress(inout headers hdr,
                 inout metadata meta,
                 inout standard_metadata_t standard_metadata) {
    action add_swtrace(switchID_t swid) {
        hdr.mri.count = hdr.mri.count + 1;
        hdr.swtraces.push_front(1);
        // According to the P4_16 spec, pushed elements are invalid, so we need
        // to call setValid(). Older bmv2 versions would mark the new header(s)
        // valid automatically (P4_14 behavior), but starting with version 1.11,
        // bmv2 conforms with the P4_16 spec.
        hdr.swtraces[0].setValid();
        hdr.swtraces[0].swid = swid;
        hdr.swtraces[0].qdepth = (qdepth_t)standard_metadata.deq_qdepth;

        hdr.ipv4.ihl = hdr.ipv4.ihl + 2;
        hdr.ipv4_option.optionLength = hdr.ipv4_option.optionLength + 8;
        hdr.ipv4.totalLen = hdr.ipv4.totalLen + 8;
    }

    table swtrace {
        actions = {
            add_swtrace;
            NoAction;
        }
        default_action = NoAction();
    }

    apply {
        if (hdr.mri.isValid()) {
            swtrace.apply();
        }
    }
}
```

Apply the trace if the MRI is valid –

```
    apply {
        /*
         * TODO: Add logic to:
         * - If hdr.mri is valid:
         *   - Apply table swtrace
         */
        if (hdr.mri.isValid()) {
            swtrace.apply();
        }
    }
}
```

Emit the results –

```
control MyDeparser(packet_out packet, in headers hdr) {
    apply {
        packet.emit(hdr.ethernet);
        packet.emit(hdr.ipv4);
        /* TODO: Emit ipv4_option, mri, and swtraces headers in order if valid */
        packet.emit(hdr.ipv4_option);
        packet.emit(hdr.mri);
        packet.emit(hdr.swtraces);
    }
}
```

Testing – We could see in the options field the MRI tag with data is listed which lists the

```
####[ Ethernet ]####
  dst      = 08:00:00:00:02:02
  src      = 08:00:00:00:02:00
  type     = IPv4
####[ IP ]####
  version  = 4
  ihl      = 10
  tos      = 0x0
  len      = 52
  id       = 1
  flags    =
  frag     = 0
  ttl      = 62
  proto    = udp
  checksum = 0x60b6
  src      = 10.0.1.1
  dst      = 10.0.2.2
  \options \
  |####[ MRI ]####
  | copy_flag = 0
  | optclass  = control
  | option    = 31
  | length    = 20
  | count     = 2
  | \swtraces \
  | |####[ SwitchTrace ]####
  | | swid     = 2
  | | qdepth   = 0
  | |####[ SwitchTrace ]####
  | | swid     = 1
  | | qdepth   = 0
####[ UDP ]####
  sport    = 1234
  dport    = 4321
  len      = 12
  checksum = 0xeb46
####[ Raw ]####
  load     = 'test'
```


7. P4 Runtime –

Installed tunnel transit –

```
# TODO build the transit rule
# TODO install the transit rule on the ingress switch
table_entry = p4info_helper.buildTableEntry(
    table_name="MyIngress.myTunnel_exact",
    match_fields={
        "hdr.myTunnel.dst_id": tunnel_id
    },
    action_name="MyIngress.myTunnel_forward",
    action_params={
        "port": SWITCH_TO_SWITCH_PORT
    })
ingress_sw.WriteTableEntry(table_entry)
print("Installed transit tunnel rule on %s" % ingress_sw.name)
```

```
print('\n----- Reading tables rules for %s -----' % sw.name)
for response in sw.ReadTableEntries():
    for entity in response.entities:
        entry = entity.table_entry
        # TODO For extra credit, you can use the p4info_helper to translate
        # the IDs in the entry to names
        table_name = p4info_helper.get_tables_name(entry.table_id)
        print('%s: ' % table_name, end=' ')
        for m in entry.match:
            print(p4info_helper.get_match_field_name(table_name, m.field_id), end=' ')
            print('%r' % (p4info_helper.get_match_field_value(m)), end=' ')
        action = entry.action.action
        action_name = p4info_helper.get_actions_name(action.action_id)
        print('->', action_name, end=' ')
        for p in action.params:
            print(p4info_helper.get_action_param_name(action_name, p.param_id), end=' ')
            print('%r' % p.value, end=' ')
        print()
```

```
# TODO Uncomment the following two lines to read table entries from s1 and s2
readTableRules(p4info_helper, s1)
readTableRules(p4info_helper, s2)
```

Testing (Ping works once we run the completed python script) we could see the counters start increasing –

```
----- Reading tables rules for s1 -----
MyIngress.ipv4_lpm: hdr.ipv4.dstAddr (b'\n\x00\x02\x02', 32) -> MyIngress.myTunnel_ingress dst_id b'd'
MyIngress.myTunnel_exact: hdr.myTunnel.dst_id b'd' -> MyIngress.myTunnel_forward port b'\x02'
MyIngress.myTunnel_exact: hdr.myTunnel.dst_id b'\xc8' -> MyIngress.myTunnel_egress dstAddr b'\x08\x00\x00\x00\x01\x11' port b'\x01'

----- Reading tables rules for s2 -----
MyIngress.ipv4_lpm: hdr.ipv4.dstAddr (b'\n\x00\x01\x01', 32) -> MyIngress.myTunnel_ingress dst_id b'\xc8'
MyIngress.myTunnel_exact: hdr.myTunnel.dst_id b'd' -> MyIngress.myTunnel_egress dstAddr b'\x08\x00\x00\x00\x02' port b'\x01'
MyIngress.myTunnel_exact: hdr.myTunnel.dst_id b'\xc8' -> MyIngress.myTunnel_forward port b'\x02'

----- Reading tunnel counters -----
s1 MyIngress.ingressTunnelCounter 100: 2 packets (196 bytes)
s2 MyIngress.egressTunnelCounter 100: 2 packets (204 bytes)
s2 MyIngress.ingressTunnelCounter 200: 2 packets (196 bytes)
s1 MyIngress.egressTunnelCounter 200: 2 packets (204 bytes)

----- Reading tunnel counters -----
s1 MyIngress.ingressTunnelCounter 100: 4 packets (392 bytes)
s2 MyIngress.egressTunnelCounter 100: 4 packets (408 bytes)
s2 MyIngress.ingressTunnelCounter 200: 4 packets (392 bytes)
s1 MyIngress.egressTunnelCounter 200: 4 packets (408 bytes)
```

8. Source Routing –

Parse the ethernet and modify the next line to parse src routing -

```
state parse_ethernet {  
    packet.extract(hdr.ethernet);  
    /*  
     * TODO: Modify the next line to select on hdr.ethernet.etherType  
     * If the value is TYPE_SRCROUTING transition to parse_srcRouting  
     * otherwise transition to accept.  
     */  
    transition select(hdr.ethernet.etherType) {  
        TYPE_SRCROUTING: parse_srcRouting;  
        default: accept;  
    }  
}
```

```
state parse_srcRouting {  
    /*  
     * TODO: extract the next entry of hdr.srcRoutes  
     * while hdr.srcRoutes.last.bos is 0 transition to this state  
     * otherwise parse ipv4  
     */  
    packet.extract(hdr.srcRoutes[next]);  
    transition select(hdr.srcRoutes[next].bos) {  
        0: parse_srcRouting;  
        1: parse_ipv4;  
    }  
}
```

Set the metadata to the port in the src routes and pop the entry

```
action srcRoute_nhop() {  
    /*  
     * TODO: set standard_metadata.egress_spec  
     * to the port in hdr.srcRoutes[0] and  
     * pop an entry from hdr.srcRoutes  
     */  
    standard_metadata.egress_spec = hdr.srcRoutes[0].port;  
    hdr.srcRoutes.pop_front(1);  
}
```

```
apply {  
    if (hdr.srcRoutes[0].isValid()) {  
        /*  
         * TODO: add logic to:  
         * - If final srcRoutes (top of stack has bos==1):  
         * - change etherType to IP  
         * - choose next hop and remove top of srcRoutes stack  
         */  
        if (hdr.srcRoutes[0].bos == 1) {  
            srcRoute_finish();  
        } else {  
            srcRoute_nhop();  
        }  
        if (hdr.ipv4.isValid()) {  
            update_ttl();  
        }  
    } else {  
        drop();  
    }  
}
```

Testing-

```
"Node: h2"
root@p4dev:/home/p4/tutorials/exercises/source_routing# ./receive.py
sniffing on eth0
got a packet
####[ Ethernet ]####
  dst      = ff:ff:ff:ff:ff:ff
  src      = 08:00:00:00:01:11
  type     = IPv4
####[ IP ]####
  version  = 4
  ihl      = 5
  tos      = 0x0
  len      = 28
  id       = 1
  flags    =
  frag     = 0
  ttl      = 59
  proto    = udp
  chksum   = 0x63ce
  src      = 10.0.1.1
  dst      = 10.0.2.2
  \options \
####[ UDP ]####
  sport    = 1234
  dport    = 4321
  len      = 8
  chksum   = 0xd328

"Node: h1"
root@p4dev:/home/p4/tutorials/exercises/source_routing# ./send.py 10.0.2.2
sending on interface eth0 to 10.0.2.2
Type space separated port nums (example: "2 3 2 2 1") or "q" to quit: 2 3 2 2 1
####[ Ethernet ]####
  dst      = ff:ff:ff:ff:ff:ff
  src      = 08:00:00:00:01:11
  type     = 0xd234
####[ SourceRoute ]####
  bos      = 0
  port     = 2
####[ SourceRoute ]####
  bos      = 0
  port     = 3
####[ SourceRoute ]####
  bos      = 0
  port     = 2
####[ SourceRoute ]####
  bos      = 0
  port     = 2
####[ SourceRoute ]####
  bos      = 1
  port     = 1
####[ IP ]####
  version  = 4
  ihl      = 5
  tos      = 0x0
  len      = 28
  id       = 1
  flags    =
  frag     = 0
  ttl      = 64
  proto    = udp
  chksum   = 0x63ce
  src      = 10.0.1.1
  dst      = 10.0.2.2
  \options \
####[ UDP ]####
  sport    = 1234
  dport    = 4321
  len      = 8
  chksum   = 0xd328
Type space separated port nums (example: "2 3 2 2 1") or "q" to quit: 
```

TTL 59 on H2 -

```
sniffing on eth0
got a packet
####[ Ethernet ]####
  dst      = ff:ff:ff:ff:ff:ff
  src      = 08:00:00:00:01:11
  type     = IPv4
####[ IP ]####
  version  = 4
  ihl      = 5
  tos      = 0x0
  len      = 28
  id       = 1
  flags    =
  frag     = 0
  ttl      = 59
  proto    = udp
  chksum   = 0x63ce
  src      = 10.0.1.1
  dst      = 10.0.2.2
  \options \
####[ UDP ]####
  sport    = 1234
  dport    = 4321
  len      = 8
  chksum   = 0xd328
```

Difference between traditional vs P4 –

Custom headers: P4 allows defining and parsing headers beyond standard protocols and allows for far greater control (e.g., MRI, telemetry fields).

Reduced dependence: P4 programs run logic directly in the data plane which allows for granular control.

Reduced TTM: P4 allows for reduced time in case of any change in protocol behavior as complete control is migrated to the software layer.

P4 allows innovation but also poses challenge due to this becoming very complex very quickly which would need specialized training for employees to operate and run the network.